Eureka Project ID: 2526Eng1001 · **Student:** Nivesh Dandyan · **Major:** BEng (CS) · **Research Mentor:** Professor Chen Ho & Dr. Rui Zhang

SIDDHI: A Self-Evolving Desktop Computer-Use Agent

with Runtime Skill Acquisition via the Learning Pipeline

Nivesh Dandyan | JC STEM Lab of Cyber Security, The University of Hong Kong | `github.com/ndpvt-web/siddhi`

INTRODUCTION

Modern AI computer-use agents (systems that can see a screen, click, type, and navigate desktop applications) represent a major step toward general-purpose automation. Yet a fundamental limitation persists: **every task starts from scratch**. Each run burns the same reasoning tokens, makes the same planning mistakes, and never builds on prior experience. There is no memory between runs.

Existing agents (Anthropic CUA and similar systems) rely entirely on the base model's parametric knowledge. They cannot learn from deployment without expensive retraining or fine-tuning.

Siddhi addresses this gap with a **runtime self-evolving skill pipeline**, the *Learning Pipeline*, that learns from its own task trajectories during normal operation, requiring no model updates. Over repeated runs, the system builds up practical knowledge: what worked, what failed, what shortcuts exist, and how to handle recurring task types with pre-compiled skill programs. The result is a measurable reduction in reasoning token use and improved task accuracy over time.

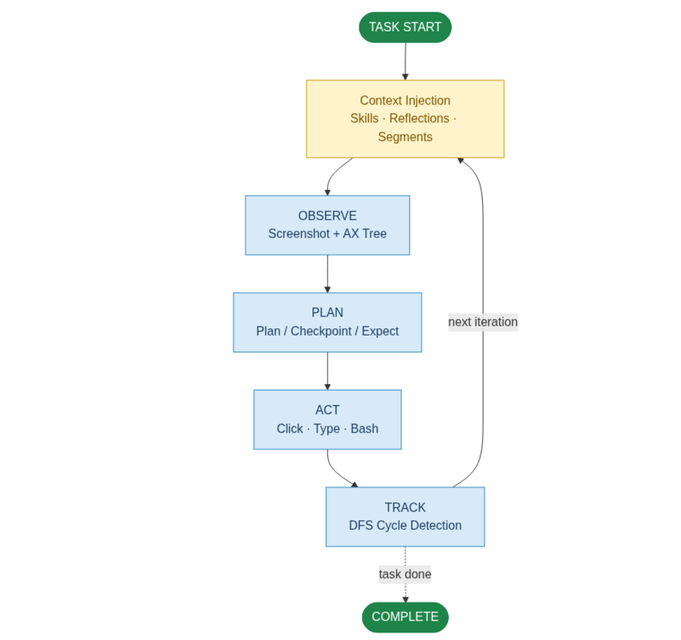
METHODOLOGY

System Architecture

Siddhi operates as a Node.js agent server (port 7888) on macOS 15+, pairing **Claude Sonnet 4.6** (default reasoning model) and **Claude Opus 4.6** (escalation on persistent failure) with a hybrid grounding system. The **macOS Accessibility API** (`capy-ax` Swift binary) provides structured UI element trees, listing every button, field, and menu item on screen as labelled data. **ShowUI-2B** (an MLX-quantized Qwen2-VL vision model, 2B parameters) handles visual coordinate refinement for elements that the Accessibility API cannot label.

The OPAR Agent Loop

Before every iteration, the Learning Pipeline injects relevant knowledge from past tasks (reflections, skills, and recurring action patterns) directly into the agent's system prompt. The agent then enters the four-phase cycle shown below, repeating until the task completes or the iteration limit is reached.



Observe: A full screenshot is captured and the complete UI element tree is fetched via the macOS Accessibility API, giving the agent a structured list of every interactive element on screen. As tasks grow longer, older screenshots from the same session are compressed into a single-sentence *"SCENE"* description, preserving what happened without the token cost of storing raw images.

Plan: Claude Sonnet 4.6 analyses the current screen state together with the injected past-task context. It produces a structured plan with numbered action steps. After verifying each sub-goal it issues a checkpoint, a confirmed safe recovery point, and predicts the next expected screen state, giving the agent a concrete target after each action. This plan-checkpoint-verify cycle keeps the agent's reasoning grounded and auditable at every step.

Act: The agent executes one tool call per iteration. Click targets are resolved through the Accessibility Tree with coordinate drift correction to handle slight layout changes between renders. Type injects text via the system automation layer, and Bash runs shell commands in a terminal. A screenshot is taken immediately after every action so the result is always verified before proceeding.

Track: Every (state, action, result) triple is appended as a node to a directed trajectory graph. DFS back-edge analysis detects loops; a zero screen-diff check detects stagnation. When a problem is detected, a four-level recovery fires: L1: try an alternative approach → L2: Cmd+Z undo → L3: rollback to the last checkpoint → L4: wipe the session state. Persistent failure beyond L3 triggers automatic model escalation from Sonnet 4.6 to Opus 4.6.

THE LEARNING PIPELINE

When a task finishes, `learnFromTrajectory()` runs asynchronously in the background, with **zero impact** on next-task startup. Raw experience is converted into structured, reusable knowledge across three layers. Each layer is more powerful than the last: observations become lessons, lessons become patterns, patterns become executable programs.

Layer 1: REFLECTIONS

The agent extracts *generalizable* lessons from the completed trajectory, e.g., *"Use Cmd+, to open preferences"*, not *"Used Cmd+, in this task"*. Each lesson is scored on four dimensions: **specificity**, **actionability**, **coherence**, **novelty**; the composite score must reach **0.6/1.0** to be stored. A TF-IDF cosine similarity check removes near-duplicates (threshold: 0.9). Relevant reflections are retrieved by keyword overlap and injected before the next task begins. *Failures from prior runs directly improve the next attempt, before it begins.*

Layer 2: SEGMENTS

Independently of reflections, the system algorithmically scans the trajectory for **recurring action sub-sequences**: groups of actions that appeared together, in order, between two checkpoints. **No model call required**; pure pattern detection. Each segment is stored in `segment_library.json` with its occurrence count and success rate. Adaptive Jaccard similarity prevents minor variations from inflating the library. This layer is the system's "proving ground": high-success patterns get promoted.

Layer 3: SKILLS (Core Innovation)

Skills are structured, reusable programs that teach the agent how to complete a specific task in a repeatable, reliable way, packaging a named recipe with preconditions, parameter slots for different files or apps, and expected outcomes. Rather than reasoning from scratch each time, the agent simply loads the relevant Skill and executes it confidently.

In Siddhi, Skills are generated *automatically at runtime*: once a Segment has proven itself reliable across multiple tasks, the system promotes it into a named Skill and makes it available for future runs. **Progressive Disclosure** keeps context lean; the agent reviews available Skills before each task and loads only what is relevant, preventing context overload.

When a matching task arrives, the agent retrieves the Skill and executes the pre-compiled steps directly, without re-reasoning from scratch. In practice, this reduces an 8-iteration run to 2.

Context Injection Budget (per task):



Skills and Reflections receive the most space as they represent the most distilled, reliable knowledge. As skills accumulate over time, they replace the need for exploratory reasoning. The context injection budget is applied *proportionally* at the start of every iteration, not as a one-time injection.

RESULTS

Phase 1: Baseline (130 Tasks)

To establish a baseline, the agent ran **130 macOS GUI tasks across 16 apps** with a completely clean learning state, with no reflections, segments, or skills. Every task was attempted purely from the base model's parametric knowledge. This measures how well the system performs *before* any runtime learning has occurred.

Metric	Value
Total tasks	130 across 16 apps
Overall success rate	**47.7%** (62/130)
Avg iterations / task	13.1
Avg task duration	169 s
Total runtime	367.5 min
Tasks hitting 15-iter cap	68

Phase 1 success rates varied widely: Mail (83%), Reminders (80%), Finder (79%) performed well; VS Code (20%), TextEdit (17%), and Contacts / Numbers / Keynote (0%) did not. The 68 failures, all hitting the 15-iteration limit, became the input for Phase 2.

Phase 2: Retry with Reflections (30 Tasks)

After Phase 1, `learnFromTrajectory()` built up the full reflection and segment corpus from all 130 tasks. The **30 hardest failures** were selected and retried, using the same tasks, instructions, and agent, but now with all accumulated knowledge injected into the system prompt before each iteration.

These 30 tasks had a 0% success rate in Phase 1. Every single one hit the 15-iteration cap without completing.

Metric	Phase 1	Phase 2	Change
Success rate	0.0%	**36.7%** (11/30)	**+36.7 pp**
Avg iterations	15.0	13.7	−1.3
Avg duration	205 s	162 s	**−43 s (−21%)**

0% → 36.7% on previously impossible tasks

Tasks the base model could *never* solve were now solved after learning from prior attempts, without any change to the underlying model.

Improvement by App

App	n	P1 SR	P2 SR	Δ SR	Iter P1→P2	Time P1→P2
Terminal	5	0%	**80%**	**+80 pp**	15.0→10.2	205s→124s
TextEdit	5	0%	**60%**	**+60 pp**	15.0→15.0	230s→195s
Notes	4	0%	**50%**	**+50 pp**	15.0→12.0	196s→141s
Safari	4	0%	**50%**	**+50 pp**	15.0→14.0	223s→158s
Finder	3	0%	0%	n/a	15.0→15.0	176s→169s
Preview	5	0%	0%	n/a	15.0→15.0	193s→171s
Calendar	3	0%	0%	n/a	15.0→15.0	196s→182s

Improvement by Complexity

Complexity	n	P2 SR	P1 Iter	P2 Iter	Δ Iter	P1 Time	P2 Time
Easy	7	**57.1%**	15.0	12.1	**−2.9**	199s	143s
Medium	15	33.3%	15.0	14.1	−0.9	208s	165s
Hard	8	25.0%	15.0	14.1	−0.9	201s	171s

Reflections help most on easy tasks. Injected lessons directly resolve specific UI actions such as keyboard shortcuts and correct menu paths. Hard tasks often involve unlabeled UI elements or multi-step dependencies that reflections alone cannot fully resolve, pointing to the Skills layer as the clear next step.

Case Studies

Task 34, Notes (easy)

"Search for a note containing 'meeting'."

P1: 15 iter / 239s / fail

P2: **5 iter / 52s** / success (−10 iter, −78% time)

Reflections provided the Cmd+F shortcut, cutting 10 iterations of blind menu exploration.

Task 51, TextEdit (easy)

"Save the current document as plain text."

P1: 15 iter / 215s / fail

P2: **4 iter / 58s** / success (−11 iter, −73% time)

Reflections encoded the Format → Make Plain Text menu path, avoiding repeated trial across menus.

Task 66, Terminal (medium)

"Check disk usage of the home directory with `du -sh`."

P1: 15 iter / 268s / fail

P2: **7 iter / 77s** / success (−8 iter, −71% time)

Reflections provided the exact command syntax, bypassing trial-and-error at the terminal.

Skill Injection: Token Savings (Preliminary)

A set of tasks were run multiple times to test the skill injection mechanism. After a skill graduated from the pipeline, subsequent runs of the same task began with the pre-compiled recipe injected into context, eliminating multi-iteration UI exploration entirely and replacing it with a single confident execution.

Task	Baseline	With Skill	Saving
Open google.com (Safari)	8 iter · $1.17	2 iter · $0.08	**14.6×**
Morning briefing	7 tools · $0.18	5 tools · $0.11	**39%**
Battery + reminder	4 iter · $0.15	2 iter · $0.076	**2×**
Screen analysis	$0.12	$0.07	**42%**

Up to 14.6× cheaper on best-case repeated task type

These results are from targeted manual trials. Formal at-scale benchmarking is planned as future work.

CONCLUSIONS

Siddhi shows that **runtime self-evolution is achievable** for desktop computer-use agents with no model retraining. Key contributions:

1. Automated skill generation from real task trajectories, the first desktop agent to do this at runtime
2. Trajectory graph with DFS cycle detection and checkpoint rollback, preventing runaway loops that exhaust context budgets
3. Adaptive graduation thresholds where evidence requirements scale with system maturity, preventing premature promotion
4. Hybrid AX+vision grounding combining structured accessibility data with visual coordinate refinement
5. Up to **14.6×** reduction in API cost on repeated task types

Future Work: Multi-agent skill sharing, cross-device trajectory transfer, and formal OSWorld evaluation.